



LECTURE 6: BASICS OF XV6

Understanding xv6: background

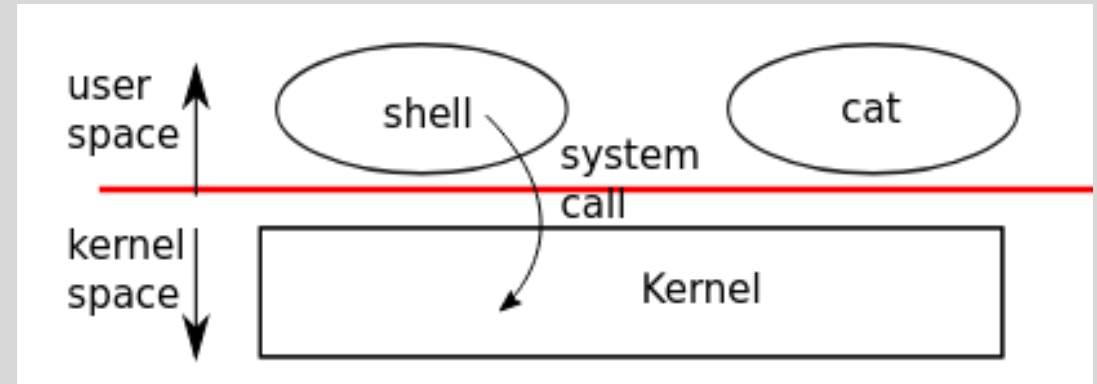
- OS enables processes stored in memory to run on CPU
 - Process code/data in main memory
 - CPU fetches, decodes, executes instructions in program code
 - Process data fetched from memory to CPU registers for faster access during instruction execution
 - Recently fetched code/data stored in CPU caches for future access (memory access is very slow compared to CPU)

How a Process is stored in main Memory?

- Process memory image consists of
 - Compiled code (CPU instructions)
 - Global/static variables (memory allocated at compile time)
 - Heap (dynamic memory allocation via, e.g., malloc) that grows on demand
 - Stack (temporary storage during function calls, e.g., local variables) that usually grows “up” towards lower addresses
 - Other things like shared libraries
- Every instruction/data has an address, used by CPU to fetch/store
 - Virtual addresses (managed by OS)
- Example: can you understand what is happening with variable “iptr”?

```
int *iptr = malloc(sizeof(int))
```

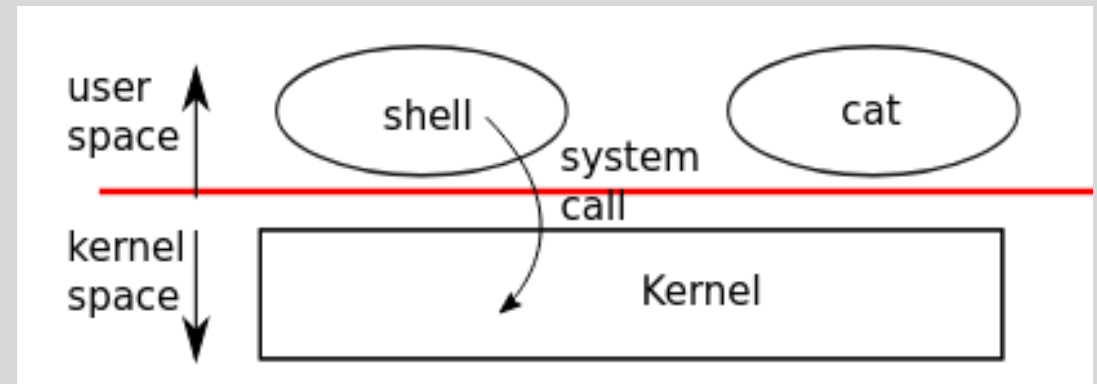
OS interfaces



- Xv6 takes the traditional form of a kernel, a special program that provides services to running programs.
- Each running program, called a process, has a memory containing instructions, data, and a stack.
- The instructions implement the program's computation. The data are the variables on which the computation acts.
- The stack organizes the program's procedure calls.
- A given computer typically has many processes but only a single kernel.

What is a System Call?

- When a process needs to invoke a kernel service, it invokes a system call, one of the calls in the operating system's interface.
- The system call enters the kernel; the kernel performs the service and returns.
- Thus a process alternates between executing in user space and kernel space.



Kernel

- The kernel uses the hardware protection mechanisms provided by the CPU to ensure that each process executing in user space can access only its own memory.
- The kernel executes with the hardware privileges required to implement these protections; user programs execute without those privileges.
- When a user program invokes a system call, the hardware raises the privilege level and starts executing a pre-arranged function in the kernel.
- The collection of system calls that a kernel provides is the interface that user programs see.

System call	Description
<code>int fork()</code>	Create a process, return child's PID.
<code>int exit(int status)</code>	Terminate the current process; status reported to <code>wait()</code> . No return.
<code>int wait(int *status)</code>	Wait for a child to exit; exit status in <code>*status</code> ; returns child PID.
<code>int kill(int pid)</code>	Terminate process PID. Returns 0, or -1 for error.
<code>int getpid()</code>	Return the current process's PID.
<code>int sleep(int n)</code>	Pause for n clock ticks.
<code>int exec(char *file, char *argv[])</code>	Load a file and execute it with arguments; only returns if error.
<code>char *sbrk(int n)</code>	Grow process's memory by n zero bytes. Returns start of new memory.
<code>int open(char *file, int flags)</code>	Open a file; flags indicate read/write; returns an fd (file descriptor).
<code>int write(int fd, char *buf, int n)</code>	Write n bytes from buf to file descriptor fd; returns n.
<code>int read(int fd, char *buf, int n)</code>	Read n bytes into buf; returns number read; or 0 if end of file.
<code>int close(int fd)</code>	Release open file fd.
<code>int dup(int fd)</code>	Return a new file descriptor referring to the same file as fd.
<code>int pipe(int p[])</code>	Create a pipe, put read/write file descriptors in <code>p[0]</code> and <code>p[1]</code> .
<code>int chdir(char *dir)</code>	Change the current directory.
<code>int mkdir(char *dir)</code>	Create a new directory.
<code>int mknod(char *file, int, int)</code>	Create a device file.
<code>int fstat(int fd, struct stat *st)</code>	Place info about an open file into <code>*st</code> .
<code>int link(char *file1, char *file2)</code>	Create another name (file2) for the file file1.
<code>int unlink(char *file)</code>	Remove a file.

Figure 1.2: Xv6 system calls. If not otherwise stated, these calls return 0 for no error, and -1 if there's an error.

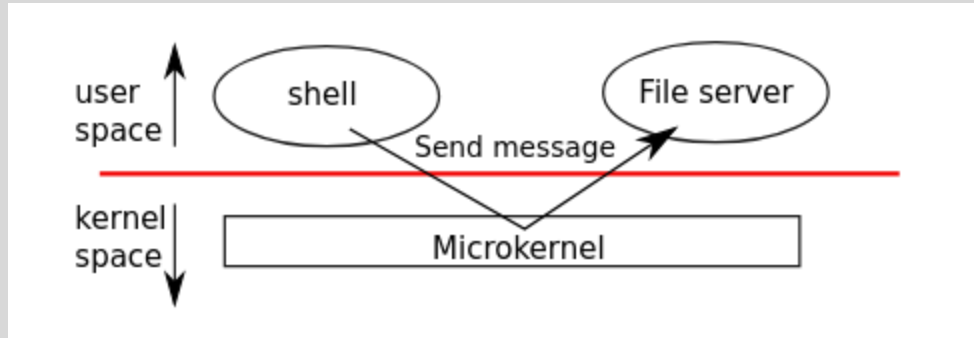
Xv6 Shell is a

- User program
- This illustrates the power of the system call interface: there is nothing special about the shell.
- It also means that the shell is easy to replace; as a result, modern Unix systems have a variety of shells to choose from, each with its own user interface and scripting features.
- The xv6 shell is a simple implementation of the essence of the Unix Bourne shell.

Monolithic Kernel

- Entire OS resides in the Kernel, and system calls runs in supervisor mode
- The OS is a single program running with full hardware privilege.
- Advantage
 - Designer need not worry which part of the OS needs h/w privilege
 - Cooperation among different part of the OS.
 - > Eg A buffer cache that can be shared both by the file system & virtual memory system
- Disadvantage
 - Interactions among different parts of OS are usually complex
 - A failure in one part of the kernel will cause the entire kernel to fail. The system must reboot to start again

Microkernel



- OS designers can minimize the amount of OS code that runs in supervisor mode and run them in user mode.
- From Figure we see that two user processes interact via IPC.
 - For example, if an application like the shell wants to read or write a file, it sends a message to the file server and waits for a response.
- Kernel interface consists of a few low-level functions.
 - Starting applications, sending messages, accessing device hardware, etc.
- In microkernel most of the operating system resides in user-level servers.

Microkernel and Monolithic Kernel

- Microkernel and monolithic operating systems share many key ideas. They implement
 - System calls,
 - Page tables,
 - Handle interrupts,
 - Support processes,
 - Use locks for concurrency control,
 - Implement a file system, etc.
- Xv6 is implemented as a monolithic kernel, like most Unix operating systems.
- Xv6 kernel interface corresponds to the operating system interface, and the kernel implements the complete operating system.
- Since xv6 doesn't provide many services, its kernel is smaller than some microkernels, but conceptually Xv6 is monolithic.



Thank You